Spec-Driven Development — Propuesta para Intrale

Fecha: 15 de marzo de 2026 | **Sprint:** SPR-031 | **Tipo:** Investigación estratégica | **Autor:** Claude (agente principal)

1. ¿Qué es Spec-Driven Development?

Spec-Driven Development (SDD) es una metodología donde **la especificación formal es el artefacto central** del ciclo de desarrollo. A diferencia de enfoques donde el código se escribe primero y la documentación se genera después, en SDD:

- **La spec se escribe ANTES del código** — define contratos, interfaces, comportamientos esperados y criterios de aceptación
- **El código se genera/implementa A PARTIR de la spec** — la spec es la fuente de verdad
- **Los tests se derivan de la spec** — cada criterio de aceptación mapea a uno o más tests
- **Los cambios se propagan desde la spec** — si cambia el requerimiento, se actualiza la spec primero

2. Estado Actual de Intrale — Mapeo de Gaps

Aspecto	Cómo funciona hoy	Gap con SDD
Definición de features	Issues en GitHub con estructura de historia de usuario (skill `/doc`)	Las historias tienen criterios de aceptación textuales pero no specs formales verificables
Contratos API	Backend Ktor con `Function` / `SecuredFunction`. No hay schema publicado	No existe OpenAPI/AsyncAPI ni spec de contrato; el frontend descubre la API leyendo el backend

Validación	Konform DSL en ViewModels, validación manual en backend	Las reglas de validación están dispersas; no hay spec compartida frontend-backend
Tests	kotlin-test + MockK, cobertura Kover ≥80%	Tests escritos post-implementación, no derivados de una spec previa
Agentes IA	Skills especializados (`/backend-dev` , `/android-dev` , etc.) con pipeline QA	Los agentes implementan desde issues textuales; sin spec formal que guíe y valide
QA	Pipeline obligatorio: QA E2E → Tester → PO	QA valida contra criterios textuales; no contra una spec ejecutable

3. Propuesta de Implementación — 4 Fases

```
┌──────────────────────────────────────────────────────────────────┐
│                    SPEC-DRIVEN DEVELOPMENT PIPELINE              │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐   │
│  │ FASE 1   │───→│ FASE 2   │───→│ FASE 3   │───→│ FASE 4   │   │
│  │ Specs de │    │ Specs de │    │ Spec de  │    │ Agentes  │   │
│  │ Contrato │    │ Comporta │    │ UI/UX    │    │ SDD-aware│   │
│  │ (API)    │    │ miento   │    │          │    │          │   │
│  └──────────┘    └──────────┘    └──────────┘    └──────────┘   │
│       │              │              │              │            │
│   OpenAPI 3.1    Gherkin/BDD     Compose Specs    Skills que    │
│   + Schemas      + Given/When    + Navigation    leen specs y   │
│   compartidos    /Then en issues  contracts      generan código │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

FASE 1 Specs de Contrato API (Semanas 1-2)

Objetivo: Definir formalmente cada endpoint del backend con OpenAPI 3.1, de modo que frontend y backend compartan una fuente de verdad.

> **Impacto inmediato:** Elimina ambigüedad en la comunicación entre capas. Los `Client[X]Service` se pueden generar o al menos validar contra la spec.

Qué hacer:

1. **Crear** `docs/api/openapi.yaml` como spec central:

```yaml
openapi: "3.1.0"
info:
  title: Intrale Platform API
  version: "1.0.0"
paths:
  /{business}/signin:
    post:
      operationId: signIn
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SignInRequest'
      responses:
        '200':
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SignInResponse'
components:
  schemas:
    SignInRequest:
      type: object
      required: [email, password]
      properties:
        email: { type: string, format: email }
        password: { type: string, minLength: 8 }
```

2. **Agregar task Gradle de validación** — `:backend:validateOpenApi` que verifique que cada `Function` registrada en Kodein tiene su correspondiente path en la spec
3. **Schemas compartidos** — Los `@Serializable` DTOs del backend y los del frontend deben derivarse de la misma spec. Opción: módulo `shared/` con los data classes comunes

Integración con el flujo actual:

Momento	Acción SDD	Responsable
Issue creado (`/doc`)	Si toca API → PO define spec del endpoint en el issue	`/po`
Pre-implementación	Agente crea/actualiza `openapi.yaml` con el nuevo endpoint	`/backend-dev`

Implementación	Backend implementa contra la spec; test valida contra schema	`/backend-dev`
QA	QA E2E valida respuestas contra schema OpenAPI	`/qa`

FASE 2 Specs de Comportamiento — BDD (Semanas 3-4)

Objetivo: Cada historia de usuario incluye especificaciones de comportamiento en formato Gherkin que se convierten directamente en tests ejecutables.

Cambio en la estructura de issues:

```
# Historia: Login con 2FA

## Criterios de aceptación (Gherkin)

```gherkin
Feature: Login con verificación 2FA

  Scenario: Login exitoso con 2FA habilitado
    Given un usuario con email "test@intrale.com" y 2FA habilitado
    When ingresa credenciales válidas
    And ingresa el código TOTP correcto
    Then el sistema retorna un JWT válido
    And redirige a la pantalla principal

  Scenario: Login fallido por código TOTP inválido
    Given un usuario con email "test@intrale.com" y 2FA habilitado
    When ingresa credenciales válidas
    And ingresa un código TOTP incorrecto
    Then el sistema muestra error "Código de verificación inválido"
    And permanece en la pantalla de verificación
```
```

Mapping a tests (automático):

```
@Test
fun `login exitoso con 2FA habilitado`() = runTest {
    // Given un usuario con email "test@intrale.com" y 2FA habilitado
    val fakeLogin = FakeDoLogin(result = DoLoginResult(requires2FA = true))
    val fake2FA = FakeDo2FAVerify(result = Do2FAVerifyResult(jwt = "valid-jwt"))
    val vm = createLoginVm(doLogin = fakeLogin, do2FA = fake2FA)

    // When ingresa credenciales válidas
    vm.onEmailChanged("test@intrale.com")
    vm.onPasswordChanged("Password123!")
    vm.onLoginClicked()
```

```
    // And ingresa el código TOTP correcto
    vm.onTotpCodeChanged("123456")
    vm.onVerifyClicked()

    // Then el sistema retorna un JWT válido
    assertEquals("valid-jwt", vm.state.jwt)
    // And redirige a la pantalla principal
    assertTrue(vm.state.navigateToHome)
}
```

> **Herramienta clave:** No necesitamos Cucumber ni frameworks BDD pesados. Basta con que los escenarios Gherkin del issue se transcriban como tests con el mapping `Given→arrange`, `When→act`, `Then→assert`. Los agentes pueden hacer esto automáticamente.

FASE 3 Specs de UI/UX y Navegación (Semanas 5-6)

Objetivo: Definir contratos de navegación y estados de UI antes de implementar pantallas.

Spec de navegación:

```
# docs/specs/navigation.yaml
screens:
  login:
    route: "/login"
    viewModel: LoginViewModel
    states:
      idle: { fields: [email, password], actions: [login, forgotPassword] }
      loading: { showProgress: true, disableActions: true }
      requires2FA: { fields: [totpCode], actions: [verify, cancel] }
      error: { message: String, recoverable: true }
    transitions:
      - from: idle → loading (on: login clicked, if: validation passes)
      - from: loading → requires2FA (on: server requires 2FA)
      - from: loading → error (on: server error)
      - from: requires2FA → home (on: 2FA verified)
```

Spec de UIState:

Cada `[Feature]UIState` data class se define primero como spec antes de codear:

```
# docs/specs/login-ui-state.yaml
LoginUIState:
  fields:
    email: { type: String, default: "", validation: "email format" }
    password: { type: String, default: "", validation: "minLength 8" }
```

```
    totpCode: { type: String, default: "" }
    isLoading: { type: Boolean, default: false }
    errorMessage: { type: String?, default: null }
    navigateToHome: { type: Boolean, default: false }
  invariants:
    - "isLoading == true → todos los campos disabled"
    - "errorMessage != null → isLoading == false"
```

FASE 4 Agentes SDD-Aware (Semanas 7-8)

Objetivo: Modificar los skills de agentes para que consuman y produzcan specs como parte integral de su flujo.

Cambios en skills existentes:

Skill	Cambio
`/doc` (historia)	Al crear/refinar un issue, incluir sección Gherkin obligatoria + referencia a spec API si aplica
`/po`	Valida que la historia tenga specs completas antes de aprobar. Nuevo checklist: ¿tiene Gherkin? ¿tiene spec API? ¿tiene spec de estados UI?
`/backend-dev`	Paso 0: leer spec OpenAPI del endpoint. Paso 1: implementar contra spec. Paso N: validar que la respuesta cumple el schema
`/android-dev`	Paso 0: leer spec de navegación y UIState. Generar data class y ViewModel skeleton desde spec
`/tester`	Genera tests directamente desde los escenarios Gherkin del issue. Cada `Scenario:` → un `@Test fun`
`/qa`	Valida E2E contra specs: respuestas API vs OpenAPI schema, estados UI vs navigation spec
`/review`	Nuevo check: ¿el código implementado es consistente con las specs del issue?

Flujo completo de un issue con SDD:

```
┌─────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐
│  /doc   │────→│   /po    │────→│  Agent   │────→│  /qa     │
│ crea    │     │ valida   │     │ implemen │     │ valida   │
```

```
  │ issue + │      │ specs    │      │ ta desde │      │ contra   │
  │ specs   │      │ completas│      │ specs    │      │ specs    │
  └─────────┘      └──────────┘      └──────────┘      └──────────┘
       │               │                │                │
       ▼               ▼                ▼                ▼
  Gherkin BDD      Spec checklist    openapi.yaml     Schema
  en el issue      aprobado          actualizado       validation
                                     tests generados   E2E vs spec
```

4. Beneficios Específicos para Intrale

Beneficio	Impacto en nuestra operación
Agentes más autónomos	Con specs formales, los agentes tienen menos ambigüedad. Reducción estimada de **re-work por malinterpretación: ~40%**
QA más precisa	QA E2E valida contra schemas, no contra expectativas textuales. Detección temprana de regresiones
Onboarding de nuevos endpoints	Un nuevo `Function` + spec OpenAPI = documentación viva. No hay drift entre docs y código
Frontend-backend desacoplados	Con schemas compartidos, el frontend puede avanzar con mocks antes de que el backend esté listo
Tests predecibles	Gherkin en issues → tests automáticos. El `/tester` no tiene que inventar escenarios
Trazabilidad completa	Issue → Spec → Código → Test → QA. Cada artefacto apunta al anterior

5. Riesgos y Mitigaciones

Riesgo	Probabilidad	Mitigación
Overhead de escribir specs	Alta	Los agentes (`/doc` , `/po`) generan las specs automáticamente. El humano solo revisa y aprueba

Specs desactualizadas	Media	Gradle task que valida consistencia spec ↔ código en CI. Si divergen, el build falla
Resistencia al cambio	Baja (somos 1+agentes)	Adopción gradual por fases. Fase 1 es la de mayor ROI y menor fricción
Complejidad en specs de UI	Media	Fase 3 es opcional para features simples. Solo obligatoria en flujos complejos con múltiples estados

6. Plan de Adopción Recomendado

> **Recomendación:** Empezar con Fase 1 (OpenAPI) + Fase 2 (Gherkin en issues) en el próximo sprint de planificación. Son las de mayor impacto con menor fricción.

Fase	Sprint estimado	Prerequisitos	Entregable
1 API Specs	SPR-032 o SPR-033	Ninguno	`docs/api/openapi.yaml` + Gradle validation task
2 BDD/Gherkin	SPR-032 o SPR-033	Ninguno	Template de issue actualizado, `/doc` genera Gherkin
3 UI Specs	SPR-034	Fase 2 estabilizada	`docs/specs/` con navigation + UIState specs
4 Agentes SDD	SPR-035	Fases 1-3 en uso	Skills actualizados para leer/producir specs

7. Ejemplo Concreto — Feature "Agregar dirección"

Para ilustrar cómo cambiaría el flujo con SDD, tomemos una feature típica:

HOY (sin SDD):

```
1. /doc crea issue → "Como cliente quiero agregar una dirección..."
2. Criterios textuales: "debe validar campos", "debe guardar en backend"
```

```
3. Agente implementa Client[Address]Service, Do[AddAddress], ViewModel, Screen
4. Agente escribe tests basados en su interpretación
5. QA valida manualmente
```

CON SDD:

```
1. /doc crea issue + spec Gherkin:
   Scenario: Agregar dirección válida
     Given estoy en la pantalla de direcciones
     When completo calle "Av. Corrientes 1234", ciudad "CABA", CP "1043"
     And presiono "Guardar"
     Then la dirección aparece en mi lista
     And recibo confirmación "Dirección agregada"

   Scenario: Validación de campos obligatorios
     Given estoy en el formulario de nueva dirección
     When dejo el campo calle vacío
     And presiono "Guardar"
     Then veo error "La calle es obligatoria"

2. /doc actualiza openapi.yaml:
   POST /{business}/client/addresses
   Schema: { street: required, city: required, zip: required }

3. Agente implementa CONTRA las specs:
   - Backend valida exactamente los campos del schema
   - Frontend implementa exactamente los estados del Gherkin
   - Tests son traducciones directas de los escenarios

4. QA valida CONTRA las specs:
   - Response body matches OpenAPI schema ✓
   - Cada escenario Gherkin se ejecuta ✓
```

> **Resultado:** Menos ambigüedad → menos re-work → sprints más predecibles → mayor calidad

8. Conclusión

Spec-Driven Development no es un cambio radical para Intrale — es una **formalización de lo que ya hacemos parcialmente**. Ya tenemos:

- Historias de usuario estructuradas (`/doc`)
- Pipeline de QA obligatorio
- Agentes que pueden leer y producir artefactos
- CI/CD con validaciones automáticas

Lo que falta es el **eslabón de la spec formal** que conecte todos estos elementos. Con OpenAPI para contratos y Gherkin para comportamiento, cerramos el loop y convertimos a los agentes en implementadores más precisos y al QA en un validador más riguroso.

> **Acción recomendada:** Crear un issue de tipo "epic" para SDD e incluir la Fase 1 + Fase 2 como historias en SPR-033 o SPR-034. Estimación: 2 historias size M + actualización de 3 skills.

Reporte generado por Claude (agente principal) — Intrale Platform — 15 de marzo de 2026
Sprint SPR-031 | Investigación estratégica